

Filter, search & order

Turn `?status=todo&search=deploy&ordering=-due_date` into a safe database question.

The big idea

Users don't want every task. They want "my **high-priority** tasks due **before Friday**, most urgent first". The client asks by adding **query parameters**:

```
GET /api/tasks/?status=todo&priority=high&ordering=due_date
```

You're writing the backend half of a filter panel. `?status=todo` is like `df[df["status"] == "todo"]` in pandas.

What your function receives

The tests call `filter_tasks(queryset, params)`. `queryset` holds these four tasks:

```
# id  title          status      priority  due_date
#  1  Deploy API      todo        high      2025-03-01
#  2  Write docs      done        low       2025-02-01
#  3  Fix deploy script todo        low       (none)
#  4  Answer email    in_progress medium     2025-04-01

params = {"status": "todo"} # a dict, like request.query_params
```

What it must return

A **queryset**. The tests compare its titles, in order, for each **params**:

```
{}
```

```
{ "status": "todo" }
```

```
{ "status": "todo", "priority": "low" }
```

```
{ "search": "DEPLOY" }
```

```
{ "due_before": "2025-03-01" }
```

```
{ "due_before": "next friday" }
```

```
{ "ordering": "-title" }
```

```
{ "ordering": "status" }
```

```
{ "status": "", "search": "" }
```

```
# all 4, in id order (test 1)
```

```
# Deploy API, Fix deploy script
```

```
# Fix deploy script (test 4)
```

```
# Deploy API, Fix deploy script
```

```
# Deploy API, Write docs (test 6)
```

```
# raises ValueError (test 7)
```

```
# Write docs, Fix deploy script, ...
```

```
# not allowed: id order (test 9)
```

```
# empty = ignored: all 4 (test 10)
```

New words

query parameters

The `name=value` pairs after `?`, joined by `&`.

query_params

Where DRF puts them: a dict-like `request.query_params`.

queryset

Which rows to fetch, and in what order. Django writes the SQL.

filter

Keep only matching rows. Several filters mean AND.

lookup

A word after `__`, like `title__icontains`, that changes the comparison.

ordering

The sort: `"title"` is A to Z, `"-title"` is Z to A.

whitelist

A fixed list of allowed values; anything else is ignored.

Tool: `queryset.filter(field=value)`

Gives back a **new**, narrower queryset, so store it: `qs = qs.filter(...)`, like `df = df[...]`.

```
from sandbox.models import Task
Task.objects.create(title="Buy milk", status="done", priority="low")
Task.objects.create(title="Book dentist", status="todo", priority="high")
Task.objects.create(title="Call Milo", status="todo", priority="low")
qs = Task.objects.all()
qs = qs.filter(status="todo")
print(qs)
qs = qs.filter(priority="high")
print(qs)
```

OUTPUT

```
<QuerySet [<Task: Book dentist>, <Task: Call Milo>]>
<QuerySet [<Task: Book dentist>]>
```

Tools: lookups and order_by

A lookup goes after the field with **two** underscores. - in `order_by` sorts backwards.

```
Task.objects.create(title="Buy milk", due_date=date(2024, 5, 10))
Task.objects.create(title="Book dentist", due_date=date(2024, 6, 1))
Task.objects.create(title="Call Milo")
print(Task.objects.filter(title__icontains="MIL"))
print(Task.objects.filter(due_date__lte=date(2024, 5, 31)))
print(Task.objects.order_by("-title"))
```

OUTPUT

```
<QuerySet [<Task: Buy milk>, <Task: Call Milo>]>
<QuerySet [<Task: Buy milk>]>
<QuerySet [<Task: Call Milo>, <Task: Buy milk>, <Task: Book dentist>]>
```

Tool: `params.get("name")`, no `KeyError`

```
params = {"colour": "red", "size": ""}  
print(params.get("colour"))  
print(params.get("size"))  
print(params.get("shape"))  
if params.get("size"):  
    print("size was given")  
else:  
    print("no size, skip it")
```

OUTPUT

red

None

no size, skip it

Tools: `date.fromisoformat` and `in`

`date.fromisoformat` is called on `date` itself; bad text raises `ValueError`.

```
print(date.fromisoformat("2024-12-25"))

SIZES = {"small", "large"}
print("large" in SIZES, "huge" in SIZES, None in SIZES)

print(date.fromisoformat("tomorrow"))
```

OUTPUT

```
2024-12-25
True False False
ValueError: Invalid isoformat string: 'tomorrow'
```

Exact match vs lookup

field=value

- `status="todo"`
- The value must match **exactly**
- `"Todo"` would not match `"todo"`
- Use for: `status`, `priority`

field__lookup=value

- `title__icontains="deploy"`
- Changes **how** it compares
- **Two** underscores: one gives `FieldError`
- Use for: `search`, `due_before`

Pattern: one if per parameter

The same shape on a plain list: each `if` narrows the result and stores it back.

```
fruits = ["apple", "banana", "cherry", "avocado"]
params = {"starts_with": "a", "max_length": ""}
if params.get("starts_with"):
    fruits = [f for f in fruits if f.startswith(params["starts_with"])]
if params.get("max_length"):
    fruits = [f for f in fruits if len(f) <= int(params["max_length"])]
print(fruits)
```

OUTPUT

```
['apple', 'avocado']
```

Your task, step by step

- 1 If `status` has a value: `queryset = queryset.filter(status=...)` → **test 2**
- 2 The same for `priority`; together they mean AND → **tests 3-4**
- 3 If `search` has a value, filter with `title__icontains=` → **test 5**
- 4 `due_before: date.fromisoformat(...)`, then `due_date__lte=` → **tests 6-7**
- 5 If `ordering` is in `ALLOWED_ORDERING`, sort by it; else by `"id"` → **tests 8-9**
- 6 Return the sorted queryset → **tests 1-10**

Common mistakes

- `queryset.filter(...)` alone does nothing: store it, `queryset = ...`
- `params["status"]` raises `KeyError` when missing: use `.get`
- `title_icontains` with one underscore gives `FieldError`
- Any ordering straight into `order_by` crashes on unknown fields
- `order_by` also returns a new queryset: return its result

Where you'll use this

A task board has a filter panel: status, priority, a search box and "sort by due date". Each click changes the query string, and your code turns it into one queryset.

- A whitelist blocks `?ordering=owner__password`, which could leak secrets
- Unknown fields would crash with a `500` and reveal your field names
- `django-filter`, `SearchFilter` and `OrderingFilter` do this for you in real projects

Check yourself

1. Why store the result: `queryset = queryset.filter(...)`?
2. Which lookup finds "Deploy" when the user searches "DEPLOY"?
3. Why does test 6 leave out a task with no due date?
4. Why check `?ordering=` against a whitelist?

Keep learning

[Django ORM and QuerySets](#)

Django Girls Tutorial · tutorial · beginner. Start here: try `filter()`, `__contains` lookups and `order_by()` in the shell, much like filtering a pandas DataFrame.

[Making queries](#)

Django docs · docs · beginner. Explains that querysets are lazy and can be chained, which is why you can add one filter at a time.

[QuerySet API reference: field lookups](#)

Django docs · docs · intermediate. Reference for `icontains`, `lte` and every other lookup, plus `order_by` with a minus sign for descending order.

[datetime: date.fromisoformat](#)

Python docs · docs · intermediate. How to turn a '2025-01-31' string from the URL into a real date, and the `ValueError` you get on bad input.

[Filtering](#)

Django REST framework · docs · intermediate. How DRF offers the same search and ordering features built in, once you understand the manual version.